

Informe de Revisión — Proyecto tienda

Revisión técnica

03/08/2026

Índice

1	Alcance	1
2	Arquitectura	1
3	Seguridad	2
4	Sincronización (<code>syncmanager.cpp</code>) — riesgos de pérdida de datos	3
5	VeriFactu — cumplimiento fiscal (riesgo de producción)	3
6	Calidad de código	4
7	Repositorio	4
8	Plan de arreglo por prioridad	4
8.1	Fase 1 — Crítico (seguridad + cumplimiento fiscal)	4
8.2	Fase 2 — Alto (pérdida de datos multi-tienda)	5
8.3	Fase 3 — Medio (robustez)	5
8.4	Fase 4 — Bajo (limpieza y calidad)	5
8.5	Orden y riesgos de ejecución	5

1 Alcance

Revisión completa del TPV (punto de venta) en Qt5/C++ con MySQL/MariaDB.

Nota: a pesar de la ruta `AndroidStudioProjects`, **no es un proyecto Android**. Es un TPV Qt/C++ con ~318 archivos trackeados, ~90 módulos, capa central `base_datos.cpp` (122 KB), sincronización con la nube y cumplimiento VeriFactu. Compila (hay binario actualizado el 03/08/2026 en build/) pero **sin tests ni CI**.

2 Arquitectura

Hallazgo	Descripción	Ubicación
God-object baseDatos	~90 métodos públicos que mezclan SQL crudo, lógica de negocio (stock, lotes) e infraestructura (backups mysqldump, rutas de imágenes). 64 archivos lo incluyen, 54 clases tienen una instancia propia.	<code>base_datos.h:18-262</code>
God-window Tienda	30+ diálogos como miembros y DDL de esquema en el constructor.	<code>tienda.cpp:102-242</code>
Global mutable conf	<code>Configuracion*</code> declarado <code>extern</code> en 2 headers, usado en 48 archivos, borrado a mano. <code>setUsuario(NULL)</code> asigna NULL int a <code>QString</code> .	<code>base_datos.h:16</code> , <code>tienda.h:57</code> , <code>tienda.cpp:884</code>
UI sin capas	La interfaz ejecuta SQL directamente.	<code>tpv.cpp:1257</code> (INSERT INTO encargos), <code>tpv.cpp:818</code>

3 Seguridad

Gravedad	Hallazgo	Ubicación
Alta	Inyección SQL por concatenación en búsquedas de texto libre (LIKE + <code>.arg()</code>).	<code>buscarProducto</code> (<code>base_datos.cpp:220</code>), <code>buscarEnTabla</code> (<code>base_datos.cpp:359</code> , alimentada desde <code>tpv.cpp:978</code>), <code>gestorencargosdialog.cpp:124</code> , <code>fabricantes.cpp:150</code> , <code>comprasventasremoto.cpp</code> (6 <code>setQuery</code>)
Alta	<code>system("cat ./ticket.txt >> " + confTicket.at(3))</code> : el path de impresora de la BD se concatena en un shell -> inyección de comandos por configuración.	<code>tpv.cpp:1095-1097</code>
Media	62+ sitios de SQL dinámico por concatenación en 18 archivos (71 con multi-línea); el patrón de consultas es inconsistente (prepared statements conviven con <code>.arg()</code>).	<code>base_datos.cpp</code> , <code>syncmanager.cpp</code> , <code>saneadorglobal.cpp...</code>
Media	Credenciales en texto plano: <code>tienda.ini</code> (root/meganizado, gitignored pero en disco), <code>tiendas.password</code> , <code>config_nube.clave</code> , clave del certificado VeriFactu.	<code>verifactudialog.cpp:45</code> , <code>login.cpp:28-37</code>

Gravedad	Hallazgo	Ubicación
Media	<code>ejecutarSentencia()</code> ejecuta SQL arbitrario; hoy solo se llama con constantes, pero es un footgun sistémico.	<code>base_datos.cpp:2035</code>
Baja	<code>qDebug</code> filtra el SQL completo construido.	<code>base_datos.cpp:945</code>

A favor: el login usa SHA-256 + salt con migración progresiva de contraseñas, bloqueo anti-fuerza-bruta, y una parte importante de las consultas críticas ya son prepared statements.

4 Sincronización (`syncmanager.cpp`) — riesgos de pérdida de datos

1. **Colisión de claves `AUTO_INCREMENT` entre tiendas:** familias, proveedores, usuarios, permisos, etc. sincronizan por id local; dos tiendas crean la misma fila con el mismo id -> se sobrescribe silenciosamente. Solo `vales` usa `vale_uuid`. (`syncmanager.cpp:368-369, 594-605`)
2. **Sin resolución de conflictos real:** “última tienda que sincroniza gana”, no el `updated_at` más nuevo. Un download sobrescribe una edición local más nueva que aún no se subió, y con `@skip_sync=1` esa edición se pierde **para siempre**. (`syncmanager.cpp:302-303, 448, 498-502`)
3. **Los borrados no propagan:** el download es solo upsert; `sync_unificaciones` nunca se sube -> las unificaciones cruzadas son código muerto. (`syncmanager.cpp:394-425, unificarmaestros.cpp:199-206`)
4. **Watermark `ultima_sync` regresivo:** retrocede 2 min por ciclo en todas las tablas inactivas cuando cualquier tabla tiene actividad. (`syncmanager.cpp:446, 553`)
5. **Red en el hilo GUI:** `sincronizar()` bloquea la UI sin timeout de lectura; con backlog grande congela el TPV. (`syncmanager.cpp:293-307`)
6. **Fuga de `@skip_sync`:** los early-returns de `descontarArticulo` dejan la variable de sesión activa -> los triggers de sync se desactivan silenciosamente. (`base_datos.cpp:546-562`)
7. **`sync_cola` crece sin límite** (las filas subidas nunca se purgan).

5 VeriFactu — cumplimiento fiscal (riesgo de producción)

#	Riesgo	Ubicación
1	El primer registro hashsea <code>Huella=000...0</code> (64 ceros) en vez de vacío -> la AEAT recomputa distinto y rechaza toda la cadena. El spec oficial usa <code>Huella= vacío</code> .	<code>base_datos.cpp:2535/2558;</code> <code>verifactuclass.cpp:180</code>
2	Facturas serie B (F1) con QR pero sin registro ni envío: el bloque se salta cuando <code>tabla != "tickets"</code> , pero la impresión muestra QR según <code>modo != 0</code> .	<code>tpv.cpp:713,</code> <code>imprimirticket.cpp:188,</code> <code>imprimirfactura.cpp:154</code>
3	RegistroAnterior usa la serie/fecha actual , no la del registro previo.	<code>verifactuclass.cpp:184-186</code>
4	Desglose a 21% hardcodeado e incoherencia <code>base+IVA != total</code> con descuentos.	<code>verifactuclass.cpp:164-170;</code> <code>totalizar.cpp:58-60</code>

#	Riesgo	Ubicación
5	Llamada AEAT dentro de la transacción: lock retenido durante el HTTP y riesgo de registro huérfano en AEAT si falla el commit.	tpv.cpp:766; verifactuclass.cpp:278-281
6	AceptadoConErrores se trata como éxito limpio; estadoEnvio=1 es engañoso en modo 2.	verifactuclass.cpp:287-289

A favor: la cadena canónica AEAT (campos, orden, UTF-8, SHA-256 hex mayúsculas) es correcta y verificada contra vectores oficiales; el QR cumple el formato AEAT (EC nivel M); el ticket + registro VeriFactu son atómicos en una transacción; el fallo de envío degrada a registro pendiente con reintento en orden.

6 Calidad de código

- **Errores ignorados:** 338 qDebug vs 35 qWarning/qCritical. vaciarTabla ignora exec() (base_datos.cpp:1566), obtenerNumeroUltimoTicket no comprueba (:1312), disminuirLote re-ejecuta sin argumentos — bug latente (base_datos.cpp:2130).
- **Tipos sistémicos en APIs:** Usuario, tickesPorCliente, registrarTickeckVerifactu, borrarUsuario (base_datos.h:43-75).
- **Duplicación:** patrón “ALTER TABLE notas” ×6 (conexion.h:74-103, base_datos.cpp:40-53), “UNION consolidado” ×11, boilerplate de bindValue ×6.
- **Bug de concurrencia real:** MonitorCaducidades reutiliza QSqlDatabase::database() del hilo principal desde un QThread -> viola Qt. El crash.log (SIGSEGV + “Driver not loaded”) apunta a esta clase de problema. (tienda.cpp:1066-1079)
- **Código muerto:** printermanager_old.*, scratch_old.cpp, dialog.*, copiaseguridad.ui (form sin clase), 200 líneas comentadas (tpv.cpp:833-887). dialogcambiocodigo.ui duplicado en tienda.pro:256-257.

7 Repositorio

- tienda.sql (566 KB dump de producción) trackeado en git — riesgo de seguridad.
- Artefactos desechables commiteados: build_output*.txt (480 KB), crash.log, schema_diff.txt, 6 bases de datos vacías *.db/*.sqlite, ~20 scripts SQL/Python one-off de análisis.
- README.md vacío, sin tests, sin CI. Instalación.txt referenciado en tienda.pro pero inexistente.

8 Plan de arreglo por prioridad

8.1 Fase 1 — Crítico (seguridad + cumplimiento fiscal)

1.1 Erradicar inyección SQL en entradas de usuario -> pasar a prepared statements: buscarEnTabla (base_datos.cpp:359, con whitelist de tabla/campo), buscarProducto (:220), buscarPorNotas (:241), gestorencargosdialog.cpp:124, fabricantes.cpp:150, tiposentradasasalidas.cpp:48, comprasventasremoto.cpp:26-100, disminuirLote (:2128), descontarArticulo (:537), clientes.cpp:266, proveedores.cpp:282-442, stock.cpp:27. Barrido final: 0 exec() con +/.arg(). *Verificación:* búsqueda con ' OR 1=1 --.

1.2 Eliminar system() de impresión (tpv.cpp:1095-1097) -> QFile/QIODevice append o QProcess con argv limpio.

1.3 VeriFactu huella del primer registro (`base_datos.cpp:2535/2558`) -> usar huella anterior **vacía** en el primer registro de cada serie; eliminar el workaround de `verifactuclass.cpp:180`. *Verificación:* vector oficial AEAT “Caso 1”.

1.4 VeriFactu serie B -> decidir: registrar serie B en `verifactu_logs` o no imprimir QR en F1; corregir `RegistroAnterior` (`verifactuclass.cpp:184-186`). *Verificación:* factura F1 con fila en `verifactu_logs` y QR validable.

8.2 Fase 2 — Alto (pérdida de datos multi-tienda)

- **2.1** Migrar tablas maestras a claves UUID (patrón `vales`) para evitar colisiones `AUTO_INCREMENT`.
- **2.2** Resolución de conflictos por `updated_at` (UTC): no sobrescribir lo más nuevo; re-encolar ediciones locales descartadas.
- **2.3** Propagar `DELETE` en `bajarCambios` y subir `sync_unificaciones` a la nube.
- **2.4** Fix watermark regresivo: avanzar `ultima_sync` por tabla con su propio `maxUpdate`.
- **2.5** Fix fuga de `@skip_sync` (reset garantizado en todos los caminos, RAII/transacción).

8.3 Fase 3 — Medio (robustez)

1. Sync fuera del hilo GUI + `MYSQL_OPT_READ/WRITE_TIMEOUT`.
2. `MonitorCaducidades`: crear la conexión dentro del hilo.
3. Purga periódica de `syncCola`.
4. Revisar errores silenciosos y sustituir `QDebug(lastError())` por logs reales.
5. VeriFactu: corregir semántica de `estadoEnvio` y de `AceptadoConErrores`.

8.4 Fase 4 — Bajo (limpieza y calidad)

1. Sacar de git `tienda.sql`, logs de build, `crash.log`, dumps, scripts one-off (con `git rm --cached` y los `.gitignore` ya existentes).
2. Borrar código muerto y el `.ui` duplicado.
3. README mínimo + `Instalación.txt`.
4. Suite QTest mínima (HashUtil, huella VeriFactu, cadena de sync) con GitHub Actions (qmake + MySQL).
5. Refactor ligero: renombrar tipos de API y consolidar el patrón “ALTER TABLE notas”.

8.5 Orden y riesgos de ejecución

1 -> 2 -> 3 -> 4. La Fase 2 toca esquema y datos (requiere backup de producción); 1.4 cambia comportamiento fiscal (requiere decisión sobre serie B). El resto es de bajo riesgo y reversible.